

Họ và tên: Nguyễn Tân Nguyên

MSV: 22021117

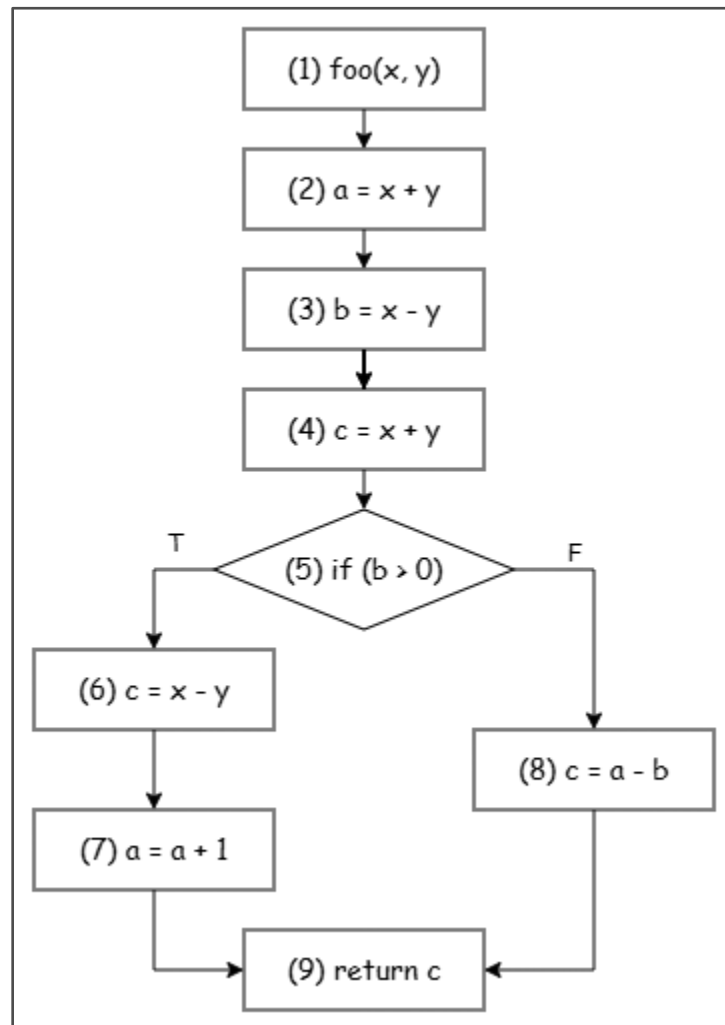
Phân tích chương trình

Mục lục

Đồ thị dòng điều khiển.....	2
Phân tích Available Expressions.....	3
Phân tích Reach Definitions.....	4
Phân tích Live Variables.....	5

```
1 int foo(int x, int y){  
2     int a = x + y;  
3     int b = x - y;  
4     int c = x + y;  
5     if (b > 0){  
6         c = x - y;  
7         a = a + 1;  
8     } else {  
9         c = a - b;  
10    }  
11    return c;  
12 }
```

Đồ thị dòng điều khiển



Phân tích Available Expressions

- Domain: $\{x + y; x - y; a + 1; a - b\}$.
- Xác định $Gen(S)$ và $Kill(S)$.

S	$Gen(S)$	$Kill(S)$
1	$\emptyset$	$\emptyset$
2	$\{x + y\}$	$\{a + 1; a - b\}$
3	$\{x - y\}$	$\{a - b\}$
4	$\{x + y\}$	$\emptyset$
5	$\emptyset$	$\emptyset$
6	$\{x - y\}$	$\emptyset$
7	$\emptyset$	$\{a + 1; a - b\}$
8	$\{a - b\}$	$\emptyset$
9	$\emptyset$	$\emptyset$

- Xác định $AE_{entry}(S)$ và $AE_{exit}(S)$.

S	$AE_{entry}(S)$	$AE_{exit}(S)$
1	$\emptyset$	$\emptyset$
2	$\emptyset$	$\{x + y\}$
3	$\{x + y\}$	$\{x + y; x - y\}$
4	$\{x + y; x - y\}$	$\{x + y; x - y\}$
5	$\{x + y; x - y\}$	$\{x + y; x - y\}$
6	$\{x + y; x - y\}$	$\{x + y; x - y\}$
7	$\{x + y; x - y\}$	$\{x + y; x - y\}$
8	$\{x + y; x - y\}$	$\{x + y; x - y; a - b\}$
9	$\{x + y; x - y\}$	$\{x + y; x - y\}$

Nhận xét:

- Mục đích của phân tích Available Expressions là để loại bỏ biểu thức con chung (Common Subexpression Elimination - CSE). Phân tích này cho trình biên dịch biết những phép tính nào là thừa vì kết quả của chúng đã có sẵn.
- Tại S4, biểu thức $x + y$ nằm trong tập $AE_{entry}(S4)$. Nó đã được tính toán tại S1 và kết quả được lưu trong biến a. Từ S1 đến S4, cả x và y đều không bị thay đổi. Từ đây, trình biên dịch sẽ không sinh mã máy để thực hiện lại phép cộng $x + y$ nữa, mà thay lệnh này bằng một phép gán trực tiếp: $c = a$.

- Tương tự, biểu thức $x - y$ nằm trong tập $AE_{entry}(S6)$. Nó đã được tính tại S3 và lưu vào biến b. Khi luồng điều khiển nhảy vào nhánh True, giá trị của x và y vẫn nguyên vẹn. Lệnh này có thể được thay thế bằng phép gán: $c = b$.

Phân tích Reach Definitions

- **Lưu ý:** Do biến x và y chỉ được truyền vào khi gọi hàm và không được định nghĩa trong hàm nên ta **không** quan tâm đến hai biến này trong phân tích.
- **Domain:**
 - Biến a: $\{(a, ?); (a, 2); (a, 7)\}$.
 - Biến b: $\{(b, ?); (b, 3)\}$.
 - Biến c: $\{(c, ?); (c, 4); (c, 6); (c, 8)\}$.
- Xác định $Gen(S)$ và $Kill(S)$.

S	Gen(S)	Kill(S)
1	$\emptyset$	$\emptyset$
2	$\{(a, 2)\}$	$\{(a, ?); (a, 7)\}$
3	$\{(b, 3)\}$	$\{(b, ?)\}$
4	$\{(c, 4)\}$	$\{(c, ?); (c, 6); (c, 8)\}$
5	$\emptyset$	$\emptyset$
6	$\{(c, 6)\}$	$\{(c, ?); (c, 4); (c, 8)\}$
7	$\{(a, 7)\}$	$\{(a, ?); (a, 2)\}$
8	$\{(c, 8)\}$	$\{(c, ?); (c, 4); (c, 6)\}$
9	$\emptyset$	$\emptyset$

- Xác định $RD_{entry}(S)$ và $RD_{exit}(S)$.

S	$RD_{entry}(S)$	$RD_{exit}(S)$
1	$\{(a, ?); (b, ?); (c, ?)\}$	$\{(a, ?); (b, ?); (c, ?)\}$
2	$\{(a, ?); (b, ?); (c, ?)\}$	$\{(a, 2); (b, ?); (c, ?)\}$
3	$\{(a, 2); (b, ?); (c, ?)\}$	$\{(a, 2); (b, 3); (c, ?)\}$
4	$\{(a, 2); (b, 3); (c, ?)\}$	$\{(a, 2); (b, 3); (c, 4)\}$
5	$\{(a, 2); (b, 3); (c, 4)\}$	$\{(a, 2); (b, 3); (c, 4)\}$
6	$\{(a, 2); (b, 3); (c, 4)\}$	$\{(a, 2); (b, 3); (c, 6)\}$
7	$\{(a, 2); (b, 3); (c, 6)\}$	$\{(a, 7); (b, 3); (c, 6)\}$
8	$\{(a, 2); (b, 3); (c, 4)\}$	$\{(a, 2); (b, 3); (c, 8)\}$
9	$\{(a, 2); (a, 7); (b, 3); (c, 6); (c, 8)\}$	$\{(a, 2); (a, 7); (b, 3); (c, 6); (c, 8)\}$

Nhận xét: Phân tích Reach Definitions không chỉ phục vụ đặc lực cho trình biên dịch trong việc tối ưu hóa mã nguồn, mà còn là nền tảng cốt lõi trong các hệ thống **kiểm chứng phần mềm** để phân tích tính đúng đắn và phát hiện lỗi. Dưới đây là 2 nhận xét quan trọng rút ra từ kết quả trên:

1. Chứng minh tính an toàn (Không có lỗi biến chưa khởi tạo)

- Nhờ ký hiệu “?”, ta có thể theo dõi sự tồn tại của các giá trị chưa khởi tạo.
- Tại bất kỳ điểm nào mà một biến cục bộ (a, b, c) được đọc (VD: b ở S5, a ở S7, c ở S9...), tập RD_{entry} tại các điểm đó **không hề chứa** ($a, ?$), ($b, ?$) hay ($c, ?$).
- Từ đó, ta chứng minh rằng hàm này **an toàn** về mặt khởi tạo biến cục bộ: mọi luồng đi đều đảm bảo biến đã được gán giá trị hợp lệ trước khi sử dụng.

2. Xây dựng chuỗi Def-use chính xác

Tập RD_{entry} giúp công cụ kiểm chứng liên kết chính xác mỗi biến được **sử dụng** ở một dòng với nơi nó được **định nghĩa**.

- Tại S7: $a = a + 1$; RD_{entry} chứa ($a, 2$). Do đó, chương trình biết chắc chắn giá trị a ở vế phải xuất phát từ kết quả của S2, tương ứng với cặp Du ($2; 7$).
- Tương tự, tại S9: $return c$; RD_{entry} chứa ($c, 6$) và ($c, 8$). Từ đây, chương trình biết rằng c mang giá trị từ S6 hoặc S8, tương ứng với các cặp Du ($6; 9$) và ($8; 9$).

Phân tích Live Variables

- Domain: $\{a; b; c; x; y\}$.
- Xác định $Gen(S)$ và $Kill(S)$.

S	Gen(S)	Kill(S)
1	$\emptyset$	$\{x; y\}$
2	$\{x; y\}$	$\{a\}$
3	$\{x; y\}$	$\{b\}$
4	$\{x; y\}$	$\{c\}$
5	$\{b\}$	$\emptyset$
6	$\{x; y\}$	$\{c\}$
7	$\{a\}$	$\{a\}$
8	$\{a; b\}$	$\{c\}$
9	$\{c\}$	$\emptyset$

- Xác định $LV_{entry}(S)$ và $LV_{exit}(S)$ theo hướng **ngược chiều** CFG.

S	$LV_{entry}(S)$	$LV_{exit}(S)$
1	$\emptyset$	$\{x; y\}$
2	$\{x; y\}$	$\{a; x; y\}$
3	$\{a; x; y\}$	$\{a; b; x; y\}$
4	$\{a; b; x; y\}$	$\{a; b; x; y\}$
5	$\{a; b; x; y\}$	$\{a; b; x; y\}$
6	$\{a; x; y\}$	$\{a; c\}$
7	$\{a; c\}$	$\{c\}$
8	$\{a; b\}$	$\{c\}$
9	$\{c\}$	$\emptyset$

Nhận xét: Phân tích Live Variables mang lại những thông tin có ý nghĩa cho việc quản lý tài nguyên và tối ưu hóa mã nguồn:

1. Xác định Dead Code

- Tại S4, lệnh $c = x + y$ có $Kill(S4) = \{c\}$. Tuy nhiên, tập $LV_{exit}(S4)$ là $\{a; b; x; y\}$ không chứa biến c .
- Điều này có nghĩa, giá trị của c được tính ở S4 **không bao giờ** được sử dụng trước khi nó bị ghi đè ở S6 hoặc S8. Phân tích này khẳng định lệnh S4 là hoàn toàn **dư thừa**.

2. Cấp phát thanh ghi

- Tại mỗi dòng lệnh, số lượng biến nằm trong tập LV_{entry} và LV_{exit} chính là số lượng thanh ghi tối thiểu mà CPU cần để lưu các giá trị trước và sau khi thực thi dòng lệnh.
- Ví dụ, tại S5, ta cần **ít nhất** 4 thanh ghi để lưu các biến $x; y; a; b$ cùng lúc.

3. Vòng đời tham số

- Tại S1, tập $LV_{entry}(S1) = \emptyset$ và tập $LV_{exit}(S1) = \{x; y\}$. Điều này chứng minh rằng hàm “sạch” và chỉ khởi tạo với các biến đầu vào qua tham số. Nếu $LV_{entry}(S1)$ không trống, điều đó ám chỉ hàm đang sử dụng một biến toàn cục hoặc trạng thái bên ngoài nào đó.